Tensor-Based Batch Fuzzing with Adaptive Perturbation Scaling for Deep Neural Networks

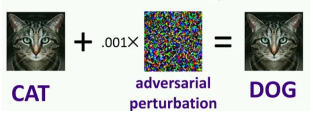
Guanqin Zhang Yulei Sui

School of Computer Science and Engineering
University of New South Wales, Sydney, Australia

ASE 2026



Neural Networks Are Everywhere, But Are They Robust?





Small perturbations on the input flip the output of a trained network.[a]

Local robustness. Perturbation set $\Phi = \{\hat{x} \mid \|\hat{x} - x\|_\infty \leq \epsilon\}$, output property Ψ (label unchanged):

$$\forall \hat{x} \in \Phi : \; f(\hat{x}) \models \Psi$$

Scaling up. Φ holds **infinitely many** inputs; models keep growing.

- No finite test suite covers Φ; today's large models make every forward pass expensive.
- **Fuzzing** searches Φ for a violating $\hat{x}$: cheap to confirm, directly actionable.
- The question: **how fast can we search Φ**, and can the search be **tensorised**?

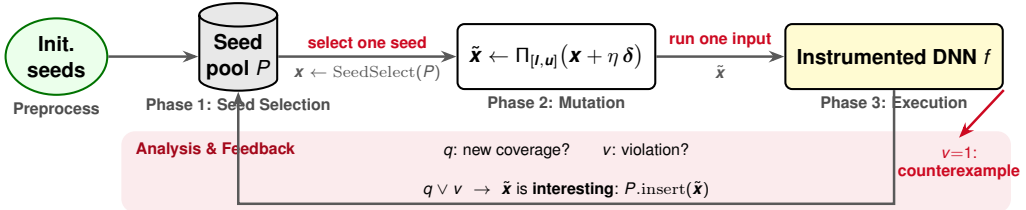
[a] Goodfellow et al., ICLR'15; Gnanasambandam et al., ICCV'21

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

UNSW SYDNEY

Existing Works Limitation: Fuzzing Is Still Done One Input at a Time

Coverage-guided DNN fuzzing (DeepHunter[1], TensorFuzz[2]): one seed, one mutant, one execution per iteration



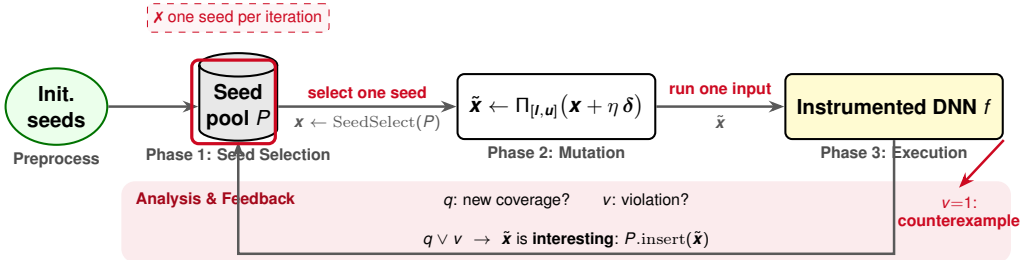
[1] Xie et al., DeepHunter: a coverage-guided fuzz testing framework for deep neural networks. ISSTA 2019.

[2] Odena et al., TensorFuzz: debugging neural networks with coverage-guided fuzzing. ICML 2019.

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

Existing Works Limitation: Fuzzing Is Still Done One Input at a Time

Coverage-guided DNN fuzzing (DeepHunter[1], TensorFuzz[2]): one seed, one mutant, one execution per iteration



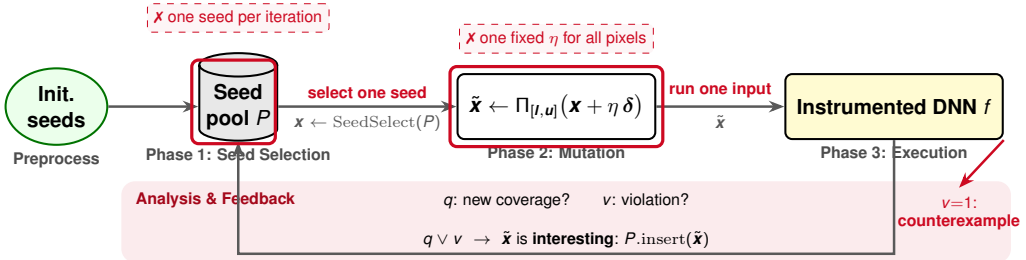
[1] Xie et al., DeepHunter: a coverage-guided fuzz testing framework for deep neural networks. ISSTA 2019.

[2] Odena et al., TensorFuzz: debugging neural networks with coverage-guided fuzzing. ICML 2019.

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

Existing Works Limitation: Fuzzing Is Still Done One Input at a Time

Coverage-guided DNN fuzzing (DeepHunter[1], TensorFuzz[2]): one seed, one mutant, one execution per iteration



[1] Xie et al., DeepHunter: a coverage-guided fuzz testing framework for deep neural networks. ISSTA 2019.

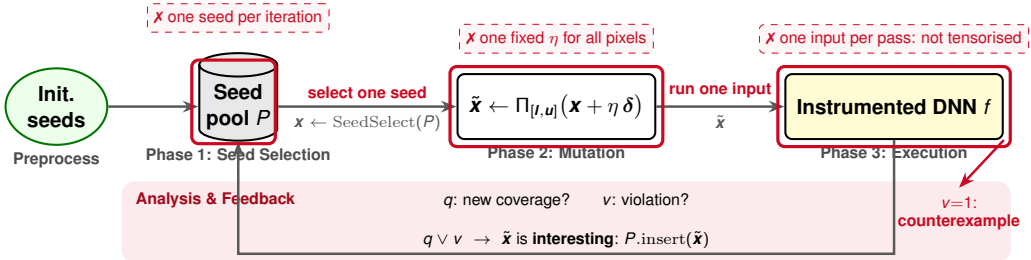
[2] Odena et al., TensorFuzz: debugging neural networks with coverage-guided fuzzing. ICML 2019.

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

UNSW SYDNEY

Existing Works Limitation: Fuzzing Is Still Done One Input at a Time

Coverage-guided DNN fuzzing (DeepHunter[1], TensorFuzz[2]): one seed, one mutant, one execution per iteration



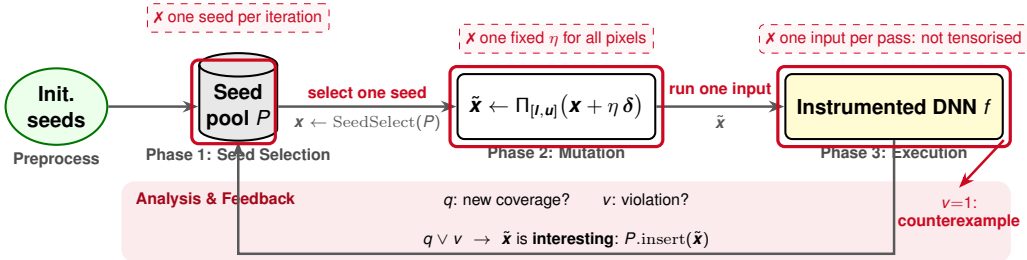
[1] Xie et al., DeepHunter: a coverage-guided fuzz testing framework for deep neural networks. ISSTA 2019.

[2] Odena et al., TensorFuzz: debugging neural networks with coverage-guided fuzzing. ICML 2019.

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

Existing Works Limitation: Fuzzing Is Still Done One Input at a Time

Coverage-guided DNN fuzzing (DeepHunter[1], TensorFuzz[2]): one seed, one mutant, one execution per iteration



Limitation

- One seed, one input per fuzzing iteration: the loop is never tensorised, so the model's batch parallelism goes unused.
- One fixed η for all pixels: the step fits no pixel well.

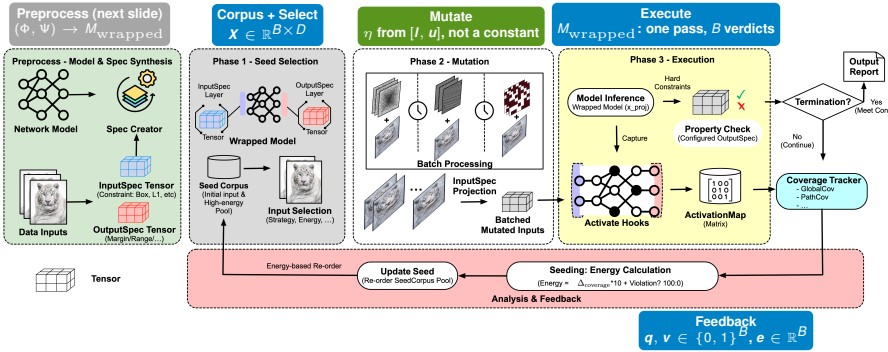
[1] Xie et al., DeepHunter: a coverage-guided fuzz testing framework for deep neural networks. ISSTA 2019.

[2] Odena et al., TensorFuzz: debugging neural networks with coverage-guided fuzzing. ICML 2019.

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

Our Approach: End-to-End Batch Loop Fuzzing Pipeline

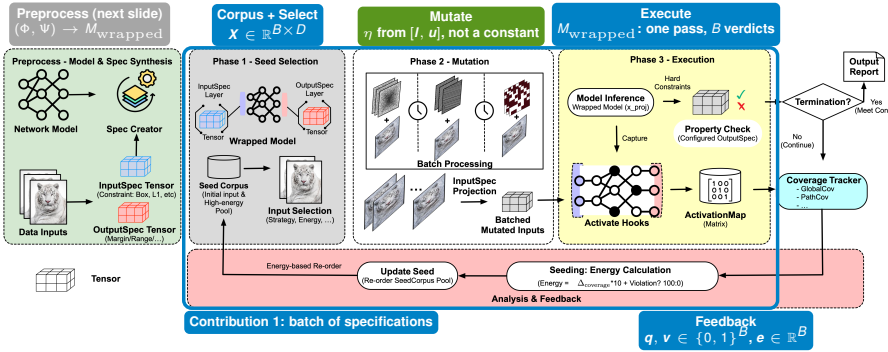
The batch loop, drawn as the tool's pipeline: preprocess, then select / mutate / execute / feedback



Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

Our Approach: End-to-End Batch Loop Fuzzing Pipeline

The batch loop, drawn as the tool's pipeline: preprocess, then select / mutate / execute / feedback

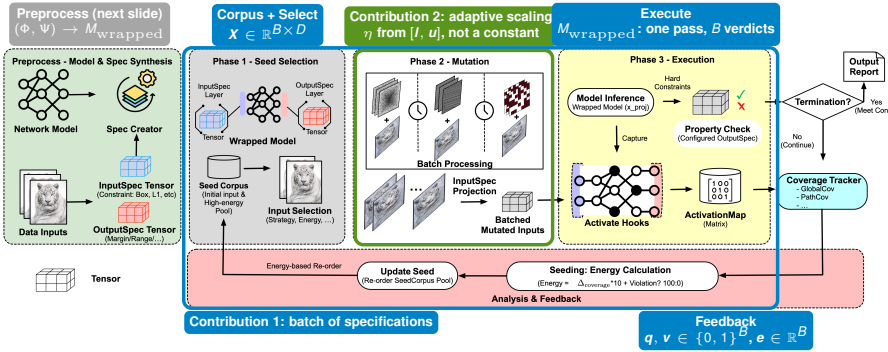


Contribution 1: fuzz a batch of specifications. Every phase runs on $X \in \mathbb{R}^{B \times D}$ as tensor ops; B multiplies throughput and **boosts test sample generation**.

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

Our Approach: End-to-End Batch Loop Fuzzing Pipeline

The batch loop, drawn as the tool's pipeline: preprocess, then select / mutate / execute / feedback



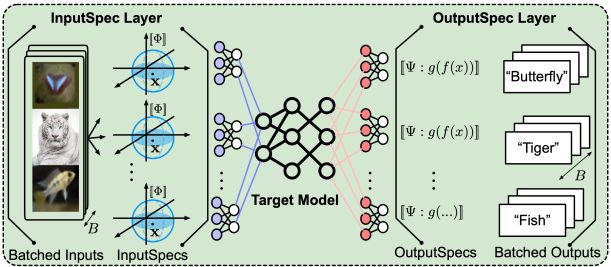
Contribution 1: fuzz a batch of specifications. Every phase runs on $X \in \mathbb{R}^{B \times D}$ as tensor ops; B multiplies throughput and **boosts test sample generation**.

Contribution 2: adaptive perturbation scaling. Step from each specification's bounds: iso $\eta = s\bar{r}$ or ani $\mathcal{S}_{b,d} = s\,(u_{b,d} - l_{b,d})$; **no hand-tuned parameter**.

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

Compile the Specification Into the Model

Phase 1 (preprocess): the specification becomes part of the model, so a batch of specifications is one forward pass



$$M_{\text{wrapped}} : \boldsymbol{X} \xrightarrow{\texttt{InputSpecLayer}(\Phi)} f(\cdot) \xrightarrow{\texttt{OutputSpecLayer}(\Psi)} v \in \{0,1\}^B$$

one batched forward pass: B inputs in, B verdicts out

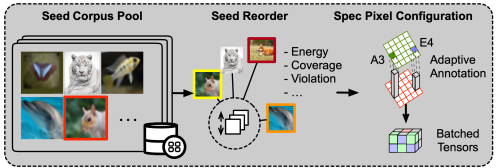
- A specification (Φ, Ψ) is just two tensor functions: **project** onto $[\boldsymbol{l}, \boldsymbol{u}]$, **check** the margin.
- So make them **non-trainable layers**: `InputSpecLayer` in front, `OutputSpecLayer` behind.
- One forward pass now takes B inputs and returns B **verdicts**; the bounds live inside the model.

For this room: the checker is instrumented into the program under test; one batched run does both.

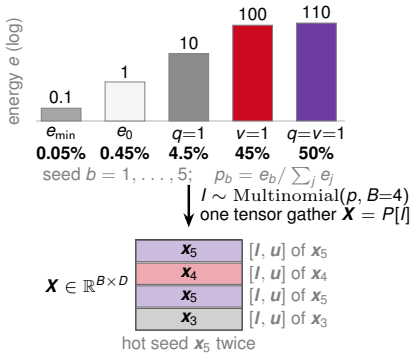
Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

Seed Selection: Energy Decides Who Gets Mutated

Phase 2: Energy-weighted sampling survives the lift, one gather draws the whole batch



- Every seed carries an **energy** e.
- One draw $\propto$ energy fills all B lanes of $\boldsymbol{X}$.



1. Score: $e_b = \max(q_b\,\alpha + v_b\,\beta,\ e_{\min})$ $\quad \beta = 10\alpha$: **a counterexample is worth $10\times$ a coverage gain**.

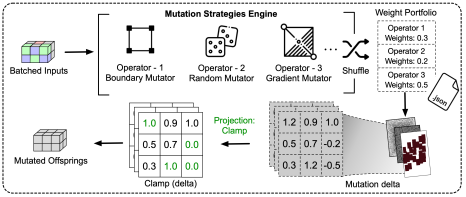
2. Sample: B seeds with probability $\propto e$, with replacement: hot seeds repeat.

3. Gather: one tensor $\boldsymbol{X} \in \mathbb{R}^{B\times D}$, each lane with its own $[\boldsymbol{l}_b, \boldsymbol{u}_b]$.

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

Mutation: Direction Is Well Studied, Perturbation Scaling Is Not

Phase 3 (mutation): conventional DNN fuzzing perturbs with a hand-picked, uniform scale η



Conventional mutation of a seed $\boldsymbol{x}$ in its box $[\boldsymbol{l}, \boldsymbol{u}]$:

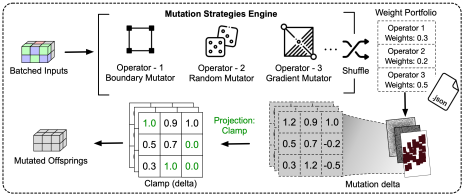
$$\tilde{\boldsymbol{x}} \;=\; \Pi_{[\boldsymbol{l},\boldsymbol{u}]}\big(\boldsymbol{x} + \eta \cdot \boldsymbol{\delta}\big)$$

- **Direction** δ = the mutation **operator**: boundary $\mathrm{sign}(\boldsymbol{r})$, random $\mathcal{N}(0, I)$, gradient $\mathrm{sign}\,\nabla_{\boldsymbol{x}}\mathcal{L}$ (FGSM, PGD, CW).
- **Perturbation scale** η: a **hyper-parameter** set by human experience (e.g. 0.01), the same for every input and every pixel.

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

Mutation: Direction Is Well Studied, Perturbation Scaling Is Not

Phase 3 (mutation): conventional DNN fuzzing perturbs with a hand-picked, uniform scale η



Conventional mutation of a seed $\boldsymbol{x}$ in its box $[\boldsymbol{l}, \boldsymbol{u}]$:

$$\tilde{\boldsymbol{x}} \;=\; \Pi_{[\boldsymbol{l},\boldsymbol{u}]}\left(\boldsymbol{x} + \eta \cdot \boldsymbol{\delta}\right)$$

- **Direction** δ = the mutation **operator**: boundary $\operatorname{sign}(\boldsymbol{r})$, random $\mathcal{N}(0, I)$, gradient $\operatorname{sign} \nabla_{\boldsymbol{x}} \mathcal{L}$ (FGSM, PGD, CW).
- **Perturbation scale** η: a **hyper-parameter** set by human experience (e.g. 0.01), the same for every input and every pixel.

Why the perturbation scale decides the search

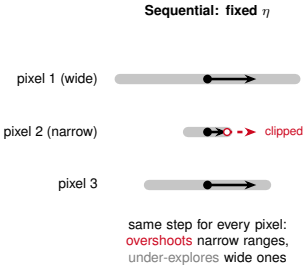
- Too small: hundreds of iterations to cross a wide range.
- Too large: leaves the box, Π **clips it**: wasted mutant.
- One η for pixels with different ranges $[l_d, u_d]$: no single value fits all.

FGSM, PGD, CW are **operators** (direction). Ours is **orthogonal**: **adaptive perturbation scaling**, from the specification bounds, per pixel, per batch; composes with any operator.

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

One Mutation Step: Sequential vs. Batch Isotropic vs. Batch Anisotropic

Each row is one **input dimension** (a pixel) d. ▬ = its feasible range $[l_d, u_d]$ from the specification; ● = current seed; **step** (arrow) = one mutation step $\eta\,\delta_d$; it must end **inside** the range, or the projection $\Pi_{[\boldsymbol{l},\boldsymbol{u}]}$ clips it.

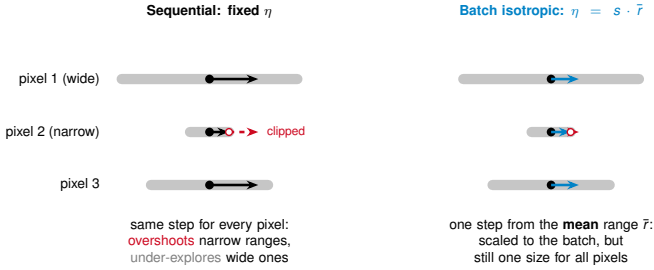


$\tilde{\boldsymbol{x}} = \Pi(\boldsymbol{x} + \eta\,\boldsymbol{\delta})$, η fixed (e.g. 0.01)
one input per iteration

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

One Mutation Step: Sequential vs. Batch Isotropic vs. Batch Anisotropic

Each row is one **input dimension** (a pixel) d. ▬ = its feasible range $[l_d, u_d]$ from the specification; ● = current seed; **step** (arrow) = one mutation step $\eta\,\delta_d$; it must end **inside** the range, or the projection $\Pi_{[\boldsymbol{l},\boldsymbol{u}]}$ clips it.



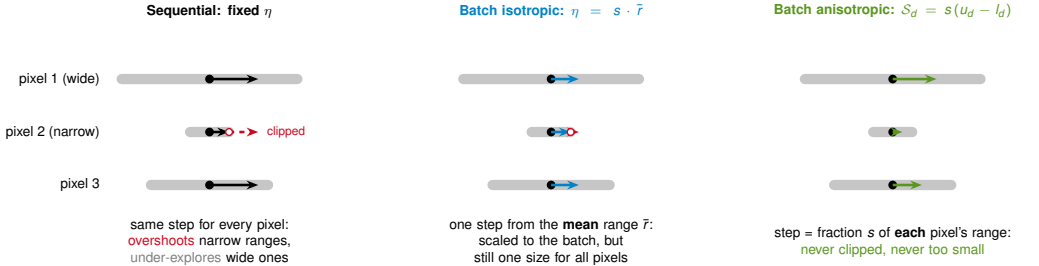
$\tilde{\boldsymbol{x}} = \Pi(\boldsymbol{x} + \eta\,\boldsymbol{\delta})$, η fixed (e.g. 0.01)
one input per iteration

$\tilde{\boldsymbol{X}} = \Pi(\boldsymbol{X} + \eta\,\Delta)$,
$\eta = s \cdot \frac{1}{BD}\sum_{b,d}(u_{b,d} - l_{b,d})$
B specs per iteration

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

One Mutation Step: Sequential vs. Batch Isotropic vs. Batch Anisotropic

Each row is one **input dimension** (a pixel) d. ▬ = its feasible range $[l_d, u_d]$ from the specification; ● = current seed; **step** (arrow) = one mutation step $\eta\,\delta_d$; it must end **inside** the range, or the projection $\Pi_{[\boldsymbol{l},\boldsymbol{u}]}$ clips it.



$\tilde{\boldsymbol{x}} = \Pi(\boldsymbol{x} + \eta\,\boldsymbol{\delta})$, η fixed (e.g. 0.01)
one input per iteration

$\tilde{\boldsymbol{X}} = \Pi(\boldsymbol{X} + \eta\,\Delta)$,
$\eta = s \cdot \frac{1}{BD}\sum_{b,d}(u_{b,d} - l_{b,d})$
B specs per iteration

$\tilde{\boldsymbol{X}} = \Pi(\boldsymbol{X} + \mathcal{S} \odot \Delta)$,
$\mathcal{S}_{b,d} = s\,(u_{b,d} - l_{b,d})$
B specs, each with its own bounds

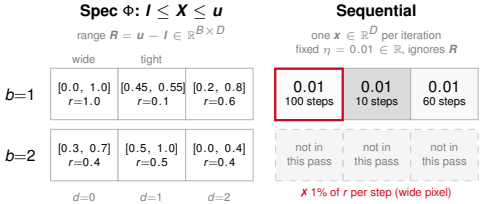
Key idea: the step size should come from the **specification's own bounds**, per pixel. Then $1/s$ steps cross every dimension, whatever its range.

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

UNSW SYDNEY

One Mutation Step on a 3-Pixel Image

B=2 **images,** *s*=0.1**. Cell: step size, steps to cross** $[l, u]$**; darker = larger share of the range**

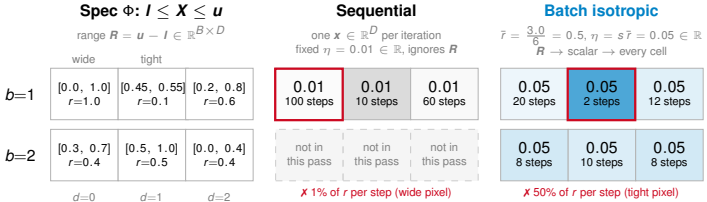


	Sequential
Mutation	$\tilde{\boldsymbol{x}} = \Pi_{[\boldsymbol{l},\boldsymbol{u}]}(\boldsymbol{x} + \eta\,\boldsymbol{\delta})$
Step size	η fixed (default 0.01)
Steps to cross wide / tight pixel	100 / 10
Specs per iteration	1
Bounds	one $[\boldsymbol{l}, \boldsymbol{u}]$
Spec heterogeneity	homogeneous

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

One Mutation Step on a 3-Pixel Image

B=2 **images,** *s*=0.1**. Cell: step size, steps to cross** $[l, u]$**; darker = larger share of the range**

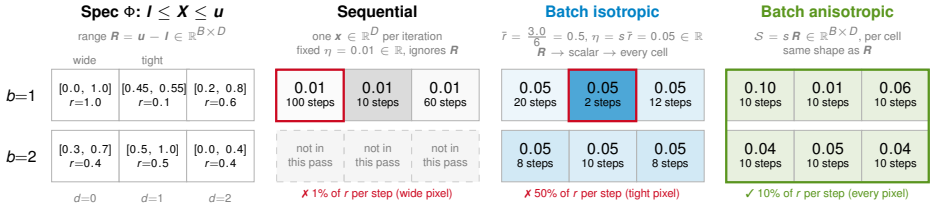


	Sequential	**Batch isotropic**
Mutation	$\tilde{x} = \Pi_{[l,u]}(x + \eta\,\delta)$	$\tilde{X} = \Pi_{[l,u]}(X + \eta\,\Delta)$
Step size	η fixed (default 0.01)	$\eta = s \cdot \bar{r}, \quad \bar{r} = \frac{1}{BD}\sum_{b,d}(u_{b,d} - l_{b,d})$
Steps to cross wide / tight pixel	100 / 10	20 / 2
Specs per iteration	1	B
Bounds	one $[l, u]$	shared $[l, u]$
Spec heterogeneity	homogeneous	homogeneous

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

One Mutation Step on a 3-Pixel Image

B=2 **images,** *s*=0.1**. Cell: step size, steps to cross** $[l, u]$**; darker = larger share of the range**



	Sequential	**Batch isotropic**	**Batch anisotropic**
Mutation	$\tilde{x} = \Pi_{[l,u]}(x + \eta\,\delta)$	$\tilde{X} = \Pi_{[l,u]}(X + \eta\,\Delta)$	$\tilde{X} = \Pi_{[l,u]}(X + S \odot \Delta)$
Step size	η fixed (default 0.01)	$\eta = s \cdot \bar{r}$, $\bar{r} = \frac{1}{BD}\sum_{b,d}(u_{b,d} - l_{b,d})$	$S_{b,d} = s\,(u_{b,d} - l_{b,d})$
Steps to cross wide / tight pixel	100 / 10	20 / 2	10 / 10 $(= 1/s)$
Specs per iteration	1	B	B
Bounds	one $[l, u]$	shared $[l, u]$	per-sample $[l_b, u_b]$
Spec heterogeneity	homogeneous	homogeneous	**heterogeneous**

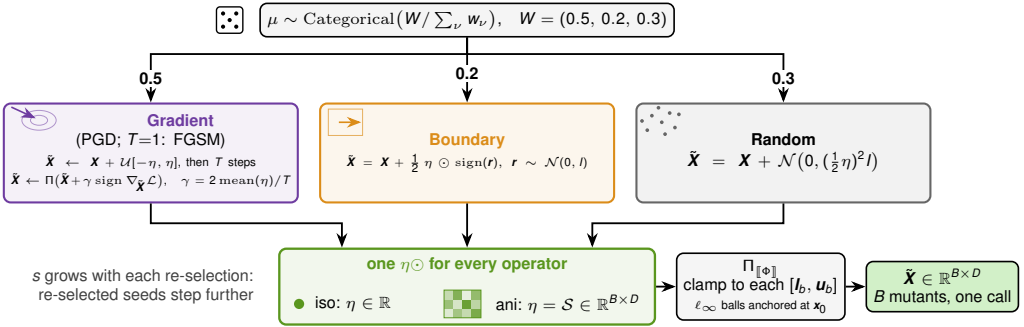
Each pixel steps by its **own** range: $1/s$ steps cross any pixel, mixed specs ride in **one batch**, one forward pass returns all B verdicts. Same 60 s budget: up to **40× throughput**, **4× more violations**.

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

UNSW SYDNEY

Mutation: One Operator per Batch, Scale Broadcast In

Direction from a weighted portfolio, magnitude from the specification (Alg. 4)

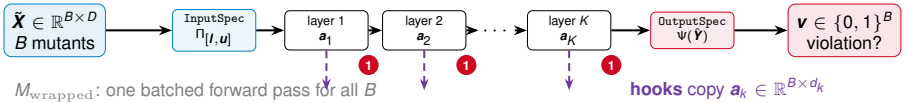


- Pick **one** operator per iteration, apply it to all B rows: forward and backward passes stay batched.
- The scale η (iso) or $\mathcal{S}$ (ani) is broadcast into **every** operator: **scaling composes with any direction**.

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

Execution: One Forward Pass, B Verdicts, Coverage for Free

Phase 4: one batched pass, then the batch statistics become the feedback



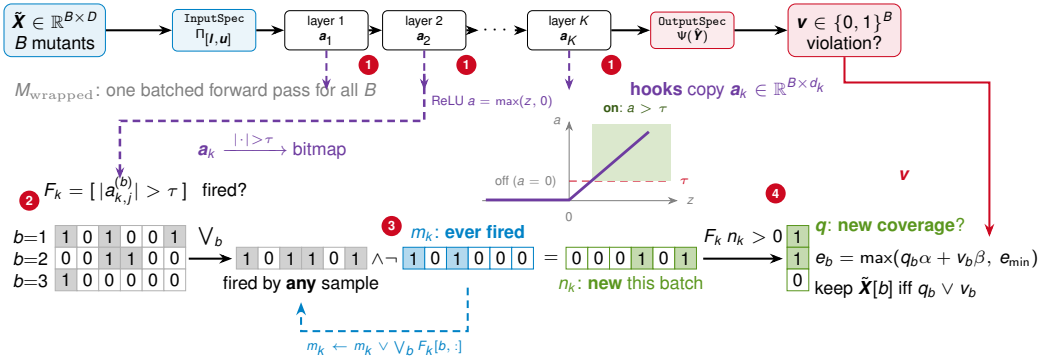
Step 1: execute the batch. One forward pass of M_{wrapped} for all B mutants; hooks copy every layer's activations $\boldsymbol{a}_k$, `OutputSpecLayer` returns the verdicts $\boldsymbol{v}$.

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

UNSW SYDNEY

Execution: One Forward Pass, B Verdicts, Coverage for Free

Phase 4: one batched pass, then the batch statistics become the feedback



Step 2: batch statistics as feedback. ② a neuron is *on* when $a > \tau$ ③ *new* = on now, never on before (m_k) ④ q_b (hit a new neuron) and v_b (violation) set the energy e_b.

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

UNSW SYDNEY

Evaluation Setup

Three benchmarks, six networks, 400+ specifications

Datasets and models

Benchmark	Networks	Specs
TrafficSigns	3 quantised CNNs	14–15 per net
Cifar100	2 ResNets (2.5M / 3.8M)	99–100 per net
TinyImageNet	1 ResNet (3.6M)	199

Specification: per-pixel box $[\boldsymbol{l}, \boldsymbol{u}]$ from VNN-COMP-style files; property Ψ = top-1 label unchanged.

Three configurations

- **Seq-Fixed**: DeepHunter loop, one input, $\eta = 0.01$.
- **Batch-Iso**: B specs per iteration, $\eta = s\,\bar{r}$.
- **Batch-Ani**: B specs per iteration, $\mathcal{S}_{b,d} = s\,(u_{b,d} - l_{b,d})$.

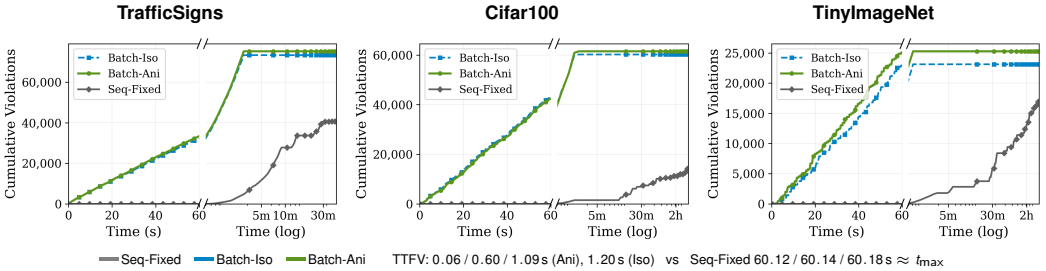
Budget: 60 s per batch run; sequential gets the same total, $B \times 60$ s.

Metrics: violations found, time to first violation, throughput (it/s).

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

Violations Over Time: Batch Finishes Before Sequential Starts

Cumulative violations. Left: first 60 s; right: full Seq-Fixed run (log time)



- First violation: batch ≈ **1 s**, Seq-Fixed ≈ **60 s**.
- Batch reaches its maximum within minutes.
- Seq-Fixed needs hours to catch up: **the gap is time, not specs**.

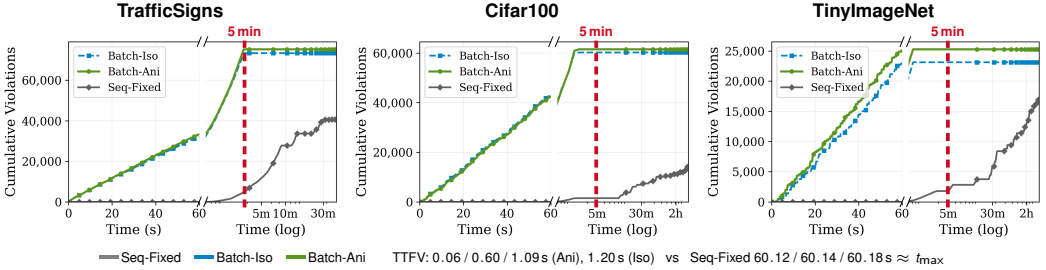
gain = $\text{viol}_{\text{batch}} - \text{viol}_{\text{seq}}$ at equal wall-clock (Batch-Ani, Table 5)

	1 min	5 min	60 min
TrafficSigns	+33K	+64K	+35K
Cifar100	+42K	+60K	+51K
TinyImageNet	+25K	+23K	+16K

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

Violations Over Time: Batch Finishes Before Sequential Starts

Cumulative violations. Left: first 60 s; right: full Seq-Fixed run (log time)



- First violation: batch ≈ **1 s**, Seq-Fixed ≈ **60 s**.
- **At 5 min batch has reached its maximum**; Seq-Fixed has barely started.
- Seq-Fixed needs hours to catch up: **the gap is time, not specs**.

gain = $\text{viol}_{\text{batch}} - \text{viol}_{\text{seq}}$ at equal wall-clock (Batch-Ani, Table 5)

	1 min	**5 min**	60 min
TrafficSigns	+33K	+64K	+35K
Cifar100	+42K	+60K	+51K
TinyImageNet	+25K	+23K	+16K

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

End-to-End Batch Fuzzing Gain

	Violations #				Throughput (it/s)			
	Seq	Iso	Ani	Ani/Seq	Seq	Iso	Ani	Ani/Seq
TrafficSigns	40,622	73,316	**75,208**	1.85×	79.6	407.3	**417.9**	5.3×
Cifar100	14,562	60,242	**61,527**	**4.2**×	15.2	516.4	**524.0**	34.5×
TinyImageNet	16,890	23,125	**25,274**	1.5×	10.9	395.1	**430.7**	**39.5**×

- **TrafficSigns**: 100% coverage everywhere, yet 1.85× more violations: **coverage is not the bottleneck**.
- Uniform ℓ_∞ bounds (**Cifar100**): $\eta = s\,\bar{r}$ fits. Per-pixel bounds (**TinyImageNet**): $\mathcal{S}_{b,d}$ wins at small s (up to +38%).
- **Larger** B: more violations overall, fewer refinements per spec: **a budget trade-off, not a tensorisation limit**.

RQ4 (5 runs, Mann–Whitney U, BH-FDR): B=10>50>99/100>199$\gg$1 (Cifar100 / TinyImageNet); Ani $\equiv$ Iso $\forall B$.

Batching multiplies (5–40× **throughput**, 1.5–4.2× **violations**); the mode does not at the default s (Iso $\approx$ Ani, within 5% over time budgets, Table 5).

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

Extended Datasets: Beyond Image Classifiers

Same pipeline, unchanged, on a text benchmark (SafeNLP) and a graph benchmark (Cora)

	Violations #					TTFV (s)			Throughput (it/s)				
	Seq	Iso	Ani	Iso/Seq	Ani/Seq	Seq	Iso	Ani	Seq	Iso	Ani	Iso/Seq	Ani/Seq
SafeNLP	32,347	**332,355**	306,300	**10.3**×	9.5×	**0.05**	0.50	0.47	316	**2,868.1**	2,672.9	**9.6**×	8.9×
Cora	1,276	**10,232**	10,085	**8.0**×	7.9×	20	0.88	**0.81**	6.0	50.8	50.8	8.5×	8.5×

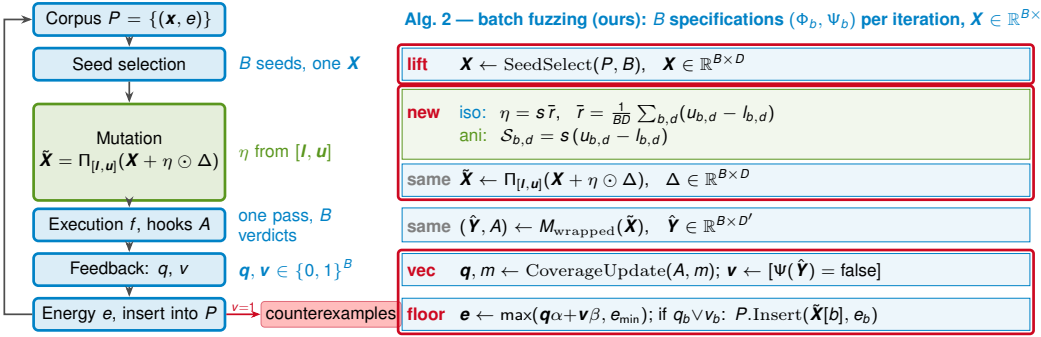
- Our tool **ACT** can adapt to different specifications.
- Batching carries over: 8–10× **violations** and ≈ 9× **throughput** on both text and graph inputs.
- Under the tensor fuzzing settings, ACT finds the the violations 1–2 orders of magnitude earlier.

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

Summary: The Batch Loop

Every phase lifted from one x to a tensor $X \in \mathbb{R}^{B\times D}$; everything else kept

Blue: lifted to a tensor. **Green**: new adaptive scale (iso scalar, ani tensor). **Red tag**: what changed in the line.



Three lines change: **lift** the seed, **add** the scale, **vectorise** the feedback. Keep Π, f and the coverage bitmap.

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering

Takeaways: End-to-End Batch Fuzzing Process

Batch: each lane = (input, $[\boldsymbol{l}, \boldsymbol{u}]$, Ψ); B verdicts per pass.

$$M_{\text{wrapped}} : \boldsymbol{X} \xrightarrow{\texttt{InputSpecLayer}(\Phi)} f(\cdot) \xrightarrow{\texttt{OutputSpecLayer}(\Psi)} v \in \{0,1\}^B$$

Scale: step = $s\times$ feasible range, per pixel, per sample.

$$\tilde{\boldsymbol{X}} = \Pi_{[\![\Phi]\!]}(\boldsymbol{X} + \mathcal{S} \odot \Delta), \qquad \mathcal{S}_{b,d} = s\,(u_{b,d} - l_{b,d})$$

$1/s$ steps cross any dimension

Evidence: up to $40\times$ throughput, $4\times$ violations, first counterexample ≤ 1.2 s.

One VNN-LIB specification (Φ, Ψ) in, B verdicts and counterexamples $\tilde{\boldsymbol{x}} \in [\![\Phi]\!]$ out; implemented in ACT.

Open source: https://github.com/SVF-tools/ACT

Thank you!

Guanqin Zhang guanqin.zhang@unsw.edu.au

Guanqin Zhang guanqin.zhang@unsw.edu.au
UNSW Sydney, School of Computer Science and Engineering